

Sequential Causal Training of Windowed PINNs for The Lorenz ODE System

Ikramul Hasan Moral
Computer Science And Engineering
imoral223489@bscse.uiu.ac.bd
0112230489

Md. Abu Bakar
Computer Science And Engineering
mbakar223200@bscse.uiu.ac.bd
0112230200

Samiur Rahman Omlan
Computer Science And Engineering
somlan223195@bscse.uiu.ac.bd
0112230195

Md. Touhidul Islam
Computer Science And Engineering
mislam223435@bscse.uiu.ac.bd
0112230435

Fariha Islam
Computer Science And Engineering
fislam223431@bscse.uiu.ac.bd
0112230431

Dr. Muhammad Nomani Kabir
Professor, Dept of CSE
United International University
kabir@cse.uiu.ac.bd

Abstract—We study a physics-informed neural network (PINN) approximation of a single full-cycle orbit of the Lorenz three-state system on $t \in [0, 13.26446]$. We trained a single network on the full interval that produces a combined state root-mean-square error (RMSE) of 2.237. To address this long horizon problem, we divide the interval in total 27 windows. Each window has a separate network and an initial state that is fixed to the predicted endpoint of the previous window. In each window, Adam reduces the residual in four stages. In each stage, we use causal weights and causal weighted loss to reduce the loss of all earlier points. Then, L-BFGS minimizes the loss using the unweighted residual mean. A numerical reference is used to monitor the training but is not used for optimization or stopping. In the seed-0 run, the windowed model gives a combined RMSE of 6.97×10^{-5} on 13,265 evaluation times and a dense-grid residual mean square of 1.87×10^{-7} . Across five seeds, the median RMSE is 8.13×10^{-5} . The full-interval and windowed runs also vary in terms of number of parameters, sampling and optimizer schedule. We therefore describe them as different configurations, not as an isolation of the effect of causal weighting. We also ran ablations that remove causal weighting, warm start or the separate network of each window.

Index Terms—physics-informed neural network, Lorenz system, causal training, time windows, ordinary differential equation, windowed PINN

I. INTRODUCTION

The Lorenz-system model is a three-state, nonlinear ordinary differential equation (ODE) that was obtained from a reduced flow model [1]. It has an initial-value problem which provides a simple test of the convergence of a learned continuous-time function from a specified trajectory over a long interval. A reference trajectory is provided by numerical integration. A PINN is trained with the residual of the ODE instead of the solution samples [2]. To train a long horizon problem using traditional PINN is not suggested by previous works [3]. It couples early and late times, but the state at a later time is determined by the previous trajectory. To overcome this ordering, Wang *et al.* suggested the use of time-dependent residual weights PINN training [3]. An initial condition can also be met by construction using a hard trial

solution [4]. We use both concepts, we split the time period in equal 27 windows and a separate network for every windows. The initial state of each window is fixed to the endpoint predicted by the previous window. The parameters of a trained window are copied to the next trainable window. We report one implementation and its saved result on $[0, 13.26446]$, which is a nearly closed orbit for the chosen initial state. The actual training goal and window handoff is then given and compared to a saved single-network training of the same initial-value problem. The state error, ODE residual, causal weights residual are also checked. The comparison is descriptive because multiple configuration options are changing simultaneously. We also ran ablation and failure cases, where we removed causal weighting, warm start or the separate network of each window.

II. PROBLEM AND BACKGROUND

A. Lorenz Equations

Our main object is a three-state nonlinear ordinary differential equation (ODE) that we need to solve. Here $x(t)$, $y(t)$ and $z(t)$ are the three states, and κ and ℓ are the physical wavenumbers. We use k only for the window index, which comes later. The Lorenz system general form [5] is:

$$\begin{aligned}\frac{dx}{dt} &= \kappa\ell \left(\frac{1}{\kappa^2 + \ell^2} - \frac{1}{\kappa^2} \right) yz, \\ \frac{dy}{dt} &= \kappa\ell \left(\frac{1}{\ell^2} - \frac{1}{\kappa^2 + \ell^2} \right) xz, \\ \frac{dz}{dt} &= \frac{\kappa\ell}{2} \left(\frac{1}{\kappa^2} - \frac{1}{\ell^2} \right) xy.\end{aligned}\tag{1}$$

Suppose the state vector is given by $u(t) = [x(t), y(t), z(t)]^T$. We put $\kappa = 2$ and $\ell = 1$ in (1) and get the three coefficients.

$$\begin{aligned}
\kappa &= 2, \quad \ell = 1, \\
a_x &= 2(1/5 - 1/4) = -1/10 = -0.1, \\
a_y &= 2(1 - 1/5) = 8/5 = 1.6, \\
a_z &= (2/2)(1/4 - 1) = -3/4 = -0.75.
\end{aligned}$$

These coefficients multiply yz , xz and xy , and this gives the reduced system we used in this experiment.

$$\dot{x} = -0.1 yz, \quad \dot{y} = 1.6 xz, \quad \dot{z} = -0.75 xy, \quad (2)$$

If we collect the three right-hand sides, we get the vector form $du/dt = f(u)$. Here f is the known ODE vector field.

$$\begin{aligned}
u &= [x, y, z]^T \in \mathbb{R}^3, \\
a &= [a_x, a_y, a_z]^T = [-0.1, 1.6, -0.75]^T, \\
\frac{du}{dt} &= f(u), \quad f: \mathbb{R}^3 \rightarrow \mathbb{R}^3, \\
f(u) &= \begin{bmatrix} a_x yz \\ a_y xz \\ a_z xy \end{bmatrix} = \begin{bmatrix} -0.1 yz \\ 1.6 xz \\ -0.75 xy \end{bmatrix}.
\end{aligned}$$

The input of f is a state vector with three components and the output is the rate of change of x , y and z in the same order. While training, we use this same function on the predicted state $\hat{u}_k(t)$.

For our experiment we used the coefficients $a = [a_x, a_y, a_z]^T = [-0.1, 1.6, -0.75]^T$. We track the system starting from an initial state [5] of $u(0) = [0.5, 0.75, 1.0]^T$ until a final time of $T = 13.26446$. On this particular rounded endpoint, the reference trajectory nearly returns to its starting state. It is very similar but not exactly closed so we call this interval a nearly closed orbit and not an exact period.

B. Related Work

In this study we are making an experimental investigation of a series of well known concepts. The approach adopted is a PINN that embeds the physics residual into the training objective [2]. We use a hard trial-function design [4] to ensure our starting state precisely at the point we want to implement it. Learning long trajectories is a well-known challenge due to the ordering of the time axis so we take inspiration from the causal residual weighting technique [3]. Lastly segmenting the timeline is based on ideas of causal sweeping and temporal decomposition [6]. Our implementation is an investigation of the performance of the individual components in this particular closed orbit of Lorenz system.

III. WINDOWED PINN

A. Network Structure

We used several modelling strategies to approximate the solution for Lorenz system trajectory over $[0, T]$, where $T = 13.26446$ is one closed loop for the chosen initial state. We first tried one network to learn the entire trajectory at once. As it failed by sticking in a plateau so we tried to split the time interval in equal segments.

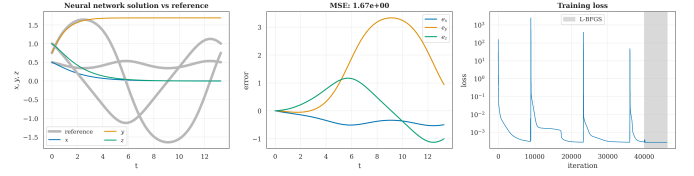


Fig. 1. Training result of a single network, entire time at once

We tried to keep the split window closer to 0.5, as small windows are better [3]. In first experiment, we used one network with 27 segmented time period to train the model sequentially. We noticed that, whenever the window is moved to the new one, the previous learning is lost or degraded. This results in significant increase in loss with RMSE of 0.350 to 0.648 over three seeds (Table I). To address these issues, we introduced WPINN (Windowed PINN). We split the total time $[0, T]$ equally into 27 windows and gave each window its own network. Here $k = 0, \dots, 26$ is the index of the 27 windows and $t_0, \dots, t_{27}$ are the 28 boundary times. Each window W_k has width $\Delta t = T/27$ and its own network parameters θ_k .

$$\begin{aligned}
t_k &= kT/27, \quad k = 0, \dots, 27, \\
W_k &= [t_k, t_{k+1}], \quad k = 0, \dots, 26, \\
\Delta t &= T/27 \approx 0.4912763, \\
N_k(\cdot; \theta_k) &: W_k \rightarrow \mathbb{R}^3.
\end{aligned}$$

Each window is approximately 0.491 time units wide.

Each window has its own network $N_k(t; \theta_k)$ with 1 input layer (t), 4 hidden layers of 60 neurons with tanh unit, and an output layer with 3 outputs. The output is not the state (x, y, z) directly. It is a correction vector, and we get the predicted state $\hat{u}_k(t)$ from the trial solution in (3). With biases, the input layer has 120 parameters, the three hidden-to-hidden layers have 3×3660 and the output layer has 183. The total counts all 27 networks.

$$\begin{aligned}
P_{\text{window}} &= 120 + 3(3660) + 183 = 11,283, \\
P_{\text{total}} &= 27P_{\text{window}} = 304,641.
\end{aligned}$$

For the model parameter initialization we used xavier-uniform weights and zero biases. The 27 networks don't share parameters but the windows has shared boundary. The completed windows predicted endpoint for the boundary is used for the initial state of the next window.

B. Window Handoff

As the Lorenz system is an initial value problem, we enforce the initial state of each window through a trial solution. Here t_k is the starting time of window k , $u_{0,k} \in \mathbb{R}^3$ is its fixed starting state and $N_k(t; \theta_k) \in \mathbb{R}^3$ is the network correction.

$$\begin{aligned}
\hat{u}_k(t) &= u_{0,k} + (t - t_k)N_k(t; \theta_k), \\
t &\in W_k, \quad k = 0, \dots, 26.
\end{aligned} \quad (3)$$

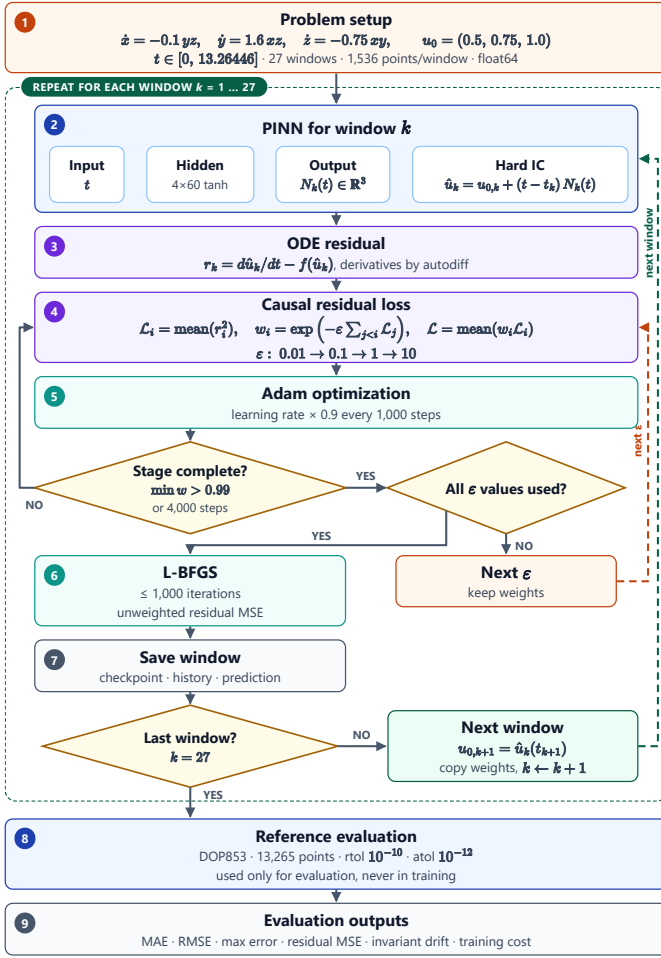


Fig. 2. Complete implementation pipeline

$$u_{0,k}, N_k(t; \theta_k), \hat{u}_k(t) \in \mathbb{R}^3, \quad t - t_k \in \mathbb{R},$$

$$\hat{u}_k(t_k) = u_{0,k} + 0 N_k(t_k; \theta_k) = u_{0,k},$$

$$u_{0,0} = [0.5, 0.75, 1.0]^\top.$$

At $t = t_k$, the second term is zero, so $\hat{u}_k(t_k) = u_{0,k}$ regardless of the network parameters [4]. The first window uses the given initial state $u_{0,0} = u(0)$, and every later window uses the predicted endpoint of the last window as its initial state.

$$u_{0,k+1} = \hat{u}_k(t_{k+1}), \quad k = 0, \dots, 25. \quad (4)$$

$$\hat{u}_{k+1}(t_{k+1}) = u_{0,k+1} = \hat{u}_k(t_{k+1}).$$

The endpoint transfer in (4) and the trial solution make the predicted values continuous at the window boundaries. But the derivatives can still change suddenly at the boundaries. Separately, window $k + 1$ starts from a copy of the weights of window k , $\theta_{k+1} \leftarrow \theta_k$ for $k = 0, \dots, 25$, and then trains its own parameters. As the starting point of the later windows is predicted, it can carry error from earlier windows.

C. Training Loss

To train the network of each window, we split the time range into $n_k = 1536$ uniform collocation points $t_{k,i}$, $i = 1, \dots, n_k$, including both endpoints.

$$n_k = 1536, \quad i = 1, \dots, n_k,$$

$$t_{k,i} = t_k + \frac{i-1}{n_k-1}(t_{k+1} - t_k),$$

$$n_{\text{unique}} = 27(1536 - 1) + 1 = 41,446,$$

$$n_{\text{window uses}} = 27 \cdot 1536 = 41,472.$$

Two windows next to each other share their boundary time. So there are 41,446 unique points in total, but 41,472 points are used in training. On the global grid, each boundary point belongs to the next window, and we also add it when we train the current window. The last window has the final endpoint. While training, all the points in a single window are passed to the network at once which gives an output of shape (1536, 3). Then we apply the trial solution to enforce the initial value. PyTorch automatic differentiation gives the rate of change of x , y and z with respect to time for all the collocation points. As $u_{0,k}$ is fixed while training, the product rule gives the derivative of (3):

$$\frac{d\hat{u}_k}{dt} = N_k + (t - t_k) \frac{\partial N_k}{\partial t}.$$

The ODE residual function compares this derivative with $f(\hat{u}_k(t))$, which is the original ODE right-hand side at the predicted state. Both terms are vectors with three components, and their difference is the residual $r_k(t)$.

$$r_k(t) = \frac{d\hat{u}_k(t)}{dt} - f(\hat{u}_k(t)) \in \mathbb{R}^3. \quad (5)$$

$$r_{k,x}(t) = \frac{d\hat{x}_k}{dt} + 0.1 \hat{y}_k \hat{z}_k,$$

$$r_{k,y}(t) = \frac{d\hat{y}_k}{dt} - 1.6 \hat{x}_k \hat{z}_k,$$

$$r_{k,z}(t) = \frac{d\hat{z}_k}{dt} + 0.75 \hat{x}_k \hat{y}_k.$$

For a batch of n_k times, each row is one collocation point and the three columns are x , y and z . Here $\mathbf{N}_k$, $\hat{\mathbf{U}}_k$, $\dot{\hat{\mathbf{U}}}_k$, $\mathbf{F}_k$ and $\mathbf{R}_k$ mean the network outputs, trial states, time derivatives, ODE right-hand sides and residuals.

$$\mathbf{t}_k = [t_{k,1}, \dots, t_{k,n_k}]^\top \in \mathbb{R}^{n_k \times 1}.$$

$$\mathbf{N}_k, \hat{\mathbf{U}}_k, \dot{\hat{\mathbf{U}}}_k, \mathbf{F}_k, \mathbf{R}_k \in \mathbb{R}^{n_k \times 3}.$$

For each ordered collocation time $t_{k,i}$, we take the mean of the three squared residual components. This gives one non-negative loss value $L_{k,i}$ for each point instead of three components:

$$L_{k,i} = \frac{1}{3} \|r_k(t_{k,i})\|_2^2$$

$$= \frac{1}{3} \left(r_{k,x}(t_{k,i})^2 + r_{k,y}(t_{k,i})^2 + r_{k,z}(t_{k,i})^2 \right) \in \mathbb{R}_{\geq 0}. \quad (6)$$

For a long horizon time dependent ODE system, residual loss alone may not be sufficient. That is why we used causal weights and causal weighted loss. The causal weight at a time depends on the accumulated residual loss at earlier times [3]. Here $S_{k,i}$ is the sum of the point losses at all earlier times of the same window. The first point has no earlier point, so $S_{k,1} = 0$ and its causal weight is one.

$$S_{k,i} = \sum_{j=1}^{i-1} L_{k,j}, \quad S_{k,1} = 0.$$

$$w_{k,i} = \exp(-\epsilon S_{k,i}) = \exp\left(-\epsilon \sum_{j=1}^{i-1} L_{k,j}\right). \quad (7)$$

$$w_{k,1} = 1, \quad 0 < w_{k,i} \leq 1, \\ \epsilon \in \{0.01, 0.1, 1, 10\}.$$

Then we multiply each point loss with its causal weight and take the mean over all n_k points. This gives one loss value for the window in the current ϵ stage:

$$\mathcal{L}_{k,\epsilon} = \frac{1}{n_k} \sum_{i=1}^{n_k} w_{k,i} L_{k,i} \in \mathbb{R}_{\geq 0}. \quad (8)$$

$$\mathbf{L}_k = [L_{k,1}, \dots, L_{k,n_k}]^T \in \mathbb{R}^{n_k},$$

$$\mathbf{w}_k = [w_{k,1}, \dots, w_{k,n_k}]^T \in \mathbb{R}^{n_k}.$$

Here k is the window, i is the current point and $j < i$ means the earlier points of i . $\mathbf{L}_k$ and $\mathbf{w}_k$ have one loss and one weight for each point. ϵ in the equation means how strongly the earlier losses reduce the later weights. We used four ϵ stages: 0.01, 0.1, 1, and 10. When the earlier losses go down, the later weights go up towards one. The weights are recomputed during training, and they are fixed (detached) while the Adam optimizer calculates the gradient. The causal weights also depend on the number and spacing of the collocation points [3].

Only using ODE residual isn't enough for long horizon system like Lorenz system. And specially for the windowed algorithm, early times should be fitted before late times, as the errors accumulates and backtracking isn't possible. Therefore, later states depend on the earlier states, if earlier error is reduced, possibly the later error can be reduced as well. Equations (6)–(8) do this with the point losses and the causal weights. The next subsection explains the stopping rules of the stages and the unweighted L-BFGS refinement steps.

D. Training Procedure

The causal loss defined above is minimized separately in each window. Figure 2 summarize the sequence across the windows. Algorithm 1 gives the training steps. In each window a Adam optimizer and StepLR scheduler is created. The rate of the scheduler is 10^{-3} and it is multiplied by 0.9 for every 1,000 updates. It is maintained for the four stages of $\epsilon = (0.01, 0.1, 1, 10)$ across the window. There is a 4,000 update limit for each stage. When the saved minimum weight

is above 0.99 or if there is an Adam update then the code will recalculate the causal weights at all collocation points at that particular window. If the minimum causal weight of all 1536 points of the window is above 0.99 after an update, then it moves to the next ϵ stage. If it is below 0.99 and it reaches the update limit then it moves to the next stage and records failure. When all four stages are completed then L-BFGS tries to minimize the loss using unweighted residual mean up to 1,000 iterations. For L-BFGS, we remove the causal weights and use the plain mean of the point losses. This is the same as the mean of all $3n_k$ squared residual components.

$$\mathcal{L}_k^{\text{raw}} = \frac{1}{n_k} \sum_i L_{k,i} = \frac{1}{3n_k} \sum_i \|r_k(t_{k,i})\|_2^2.$$

To keep track of training we use a snapshot at every 50 epoch to keep training history.

The code saves the parameters of each window, transfers the endpoint state, and copies the parameters to the next trainable window. The config file has the same configuration as all modules were, initial states, training history, next window, PyTorch, NumPy etc. A different configuration in a config is rejected. The run also outputs stage status and residual snapshots. DOP853 values are captured at set up evaluative points. They do not enter the loss, gradient, or stopping test. After training, a dedicated custom function collects all the artifacts and evaluates the final piecewise model and its residual on the collocation and evaluation grids. Finally, a summary function computes error and invariant measures from these arrays. The reported full residual is one of the evaluation.

IV. EXPERIMENTAL SETUP

A. Reference Solution

Table II shows the parameters used for the reported windowed run. We use SciPy's DOP853 [7] as the numerical reference, a relative tolerance of 10^{-10} and an absolute tolerance of 10^{-12} . It is sampled at 13,265 equally-spaced time points on a separate evaluation grid. This is a numerical reference at specified tolerances.

B. Evaluation

At each evaluation time t_j , $j = 1, \dots, n_{\text{eval}}$, we subtract the reference state from the predicted state and get the error vector $e(t_j) \in \mathbb{R}^3$. This error is not the same as the ODE residual.

$$e(t_j) = \hat{u}(t_j) - u_{\text{ref}}(t_j) \\ = [e_x(t_j), e_y(t_j), e_z(t_j)]^T \in \mathbb{R}^3, \\ j = 1, \dots, n_{\text{eval}}.$$

For the combined RMSE, we take the mean of the squared error norms over all evaluation times and then the square root.

TABLE I

ABLATION AND FAILURE-CASE RESULTS ON 13,265 EVALUATION TIMES. RMSE IS THE COMBINED STATE NORM; RESIDUAL MSE IS THE MEDIAN DENSE-GRID VALUE. CANDIDATE SEEDS ARE 0–4; NO-CAUSAL, NO-WARM-START, AND SHARED-NETWORK SEEDS ARE 1–3. BOTH FULL-INTERVAL CASES USE SEED 0. CANDIDATE SEED 0 IS THE EARLIER RUN OF RECORD; F1 IS A SEPARATE REFERENCE.

Configuration	Seeds	Median RMSE	RMSE range	Residual MSE
Candidate	5	8.13×10^{-5}	6.97×10^{-5} – 2.60×10^{-4}	1.87×10^{-7}
No causal	3	1.89×10^{-4}	1.75×10^{-4} – 5.16×10^{-4}	8.81×10^{-7}
No warm start	3	2.85×10^{-4}	1.76×10^{-4} – 2.88×10^{-4}	8.35×10^{-7}
Shared network	3	0.486	0.350–0.648	0.965
Causal single	1	0.275	–	5.23×10^{-6}
Full interval	1	2.237	–	–

Algorithm 1 Sequential causal window training

Require: Ordered windows $W_0, \dots, W_{26}$ and their uniform time points

- 1: Set $u_{0,0} \leftarrow [0.5, 0.75, 1.0]^T$
- 2: **for** $k = 0, \dots, 26$ **do**
- 3: **if** $k > 0$ **then**
- 4: Set $u_{0,k} \leftarrow \hat{u}_{k-1}(t_k)$
- 5: Initialize $\theta_k \leftarrow \theta_{k-1}$
- 6: **end if**
- 7: Add t_{k+1} to the residual points if $k < 26$
- 8: **for** $\epsilon \in (0.01, 0.1, 1, 10)$ **do**
- 9: **for** at most 4,000 Adam steps **do**
- 10: Use ordered times $t_{k,i}, i = 1, \dots, n_k$
- 11: Evaluate $N_k(t_{k,i}; \theta_k) \in \mathbb{R}^3$
- 12: Form $\hat{u}_k(t_{k,i})$ using (3)
- 13: Compute $d\hat{u}_k/dt$ by automatic differentiation
- 14: Evaluate $f(\hat{u}_k)$ and form r_k using (5)
- 15: Compute $L_{k,i}$ using (6)
- 16: Compute $w_{k,i}$ using (7); detach weights
- 17: Backpropagate (8); Adam then StepLR
- 18: **if** verified post-update $\min_i w_{k,i} > 0.99$ **then**
- 19: **break**
- 20: **end if**
- 21: **end for**
- 22: Record whether the threshold was met or capped
- 23: **end for**
- 24: Run L-BFGS on the unweighted residual loss
- 25: Save window model, endpoint state, and config record
- 26: **end for**

It is also equal to the square root of the sum of the three squared component RMSEs.

$$\begin{aligned} \text{RMSE}_2 &= \left(\frac{1}{n_{\text{eval}}} \sum_{j=1}^{n_{\text{eval}}} \|e(t_j)\|_2^2 \right)^{1/2} \\ &= (\text{RMSE}_x^2 + \text{RMSE}_y^2 + \text{RMSE}_z^2)^{1/2}. \end{aligned} \quad (9)$$

For each state $c \in \{x, y, z\}$ we also report MAE, RMSE and the maximum absolute error. E_{mean} and E_{max} are the mean

TABLE II
WINDOWED-RUN CONFIGURATION

Setting	Value
Interval; windows	[0, 13.26446]; 27 equal windows
Network per window	1–60–60–60–3, tanh
Parameters	11,283/window; 304,641 total
Initial condition; precision	$[0.5, 0.75, 1.0]^T$; float64
Uniform collocation grid	41,446 unique points
Adam ϵ stages	0.01, 0.1, 1, 10
Stage threshold; steps/stage	$\min_{1 \leq i \leq n_k} w_{k,i} > 0.99$; 4,000
Adam rate; scheduler	10^{-3} ; $\times 0.9$ per 1,000 steps
L-BFGS budget	at most 1,000 iterations/window
Warm start; seed	Initialize $\theta_k \leftarrow \theta_{k-1}$; 0
Reference grid	13,265 uniform times

and the maximum of the error norm.

$$\begin{aligned} \text{MAE}_c &= \frac{1}{n_{\text{eval}}} \sum_j |e_c(t_j)|, \\ \text{RMSE}_c &= \left(\frac{1}{n_{\text{eval}}} \sum_j e_c(t_j)^2 \right)^{1/2}, \\ \text{Max}_c &= \max_j |e_c(t_j)|, \quad c \in \{x, y, z\}, \\ E_{\text{mean}} &= \frac{1}{n_{\text{eval}}} \sum_j \|e(t_j)\|_2, \\ E_{\text{max}} &= \max_j \|e(t_j)\|_2. \end{aligned}$$

The dense-grid residual mean square is the average of the three residual components squared, taken over the times of the evaluation. So it has a factor of $1/(3n_{\text{eval}})$, but the combined state RMSE is not divided by three.

$$\text{MSE}_{r,\text{eval}} = \frac{1}{3n_{\text{eval}}} \sum_{j=1}^{n_{\text{eval}}} \|r(t_j)\|_2^2.$$

The comparison run is with the same network depth and width for the entire interval $[0, T]$. The windowed run simultaneously changes the number of networks, the grid, loss, warm start and the optimizer schedule. We measure their final output and compare them.

C. Ablation Protocol

Table I shows the ablation and failure-case runs. All runs use the same initial state, coefficients, float64 precision, interval

TABLE III
SEED-0 WINDOWED-RUN ERROR AGAINST DOP853 ON 13,265 TIMES

State	MAE	RMSE	Maximum
x	1.89×10^{-5}	2.74×10^{-5}	1.13×10^{-4}
y	2.94×10^{-5}	4.96×10^{-5}	2.88×10^{-4}
z	2.78×10^{-5}	4.05×10^{-5}	1.30×10^{-4}
State norm	5.12×10^{-5}	6.97×10^{-5}	3.09×10^{-4}

and DOP853 reference grid. The candidate is our windowed PINN from Section III, and we ran it with seeds 0–4. Seed 0 is the run we reported in Tables III and IV. For the ablations we used seeds 1–3. In the no-causal case we removed the causal weights and trained each window for 1,300 Adam steps (35,100 in total). This is close to the candidate, which used 34,768 steps on average. In the no-warm-start case we kept the candidate schedule but did not copy the parameters of the previous window. The shared-network case uses the no-causal schedule, but one network is trained for all 27 windows one after another. In the causal single-network case we used causal weights on one network for the full interval, with the same grid and budget as the full-interval run: 39,793 LHS points, 40,000 Adam steps and at most 5,000 L-BFGS iterations. Some runs were on CPU and some on GPU, so we compare Adam steps and not wall time.

D. Failure Cases

We used two failure cases. The shared-network case checks if one network can keep what it learned in the earlier windows. The causal single-network case checks if causal weighting alone can fix the error of the full-interval network. We judge a run only by its final error against DOP853. All the runs completed training, so a large error here means a poor prediction and not a stopped job. When a stage reaches the 4,000-step cap we record it, but this alone does not mean the trajectory failed. We did not use any success cutoff to pick runs, and Table I shows every completed seed.

V. RESULTS

A. Prediction Accuracy

We compare the windowed PINN with an extremely accurate numerical reference which is SciPy’s DOP853. As shown in Table III, our model achieved a combined RMSE of 6.97×10^{-5} across the 13,265 uniform times in our evaluation grid. The highest state error norm was 3.09×10^{-4} . Figure 3 plots these predictions against the reference over time. The trajectories are shown to overlap completely but the error show small peaks in the errors particularly at $t \simeq 5$. In the state-norm row, the MAE-column entry is E_{mean} , the RMSE entry is RMSE_2 , and the maximum entry is E_{max} .

We also measured how closely the model obeyed the underlying equations. The MSE of the ODE residual was very small on the evaluation grid at 1.872×10^{-7} . Also the residual MSE of windowed run on collocation grid is nearly same (1.874×10^{-7}). The DOP853 reference drifted less than 4×10^{-10} for perspective.

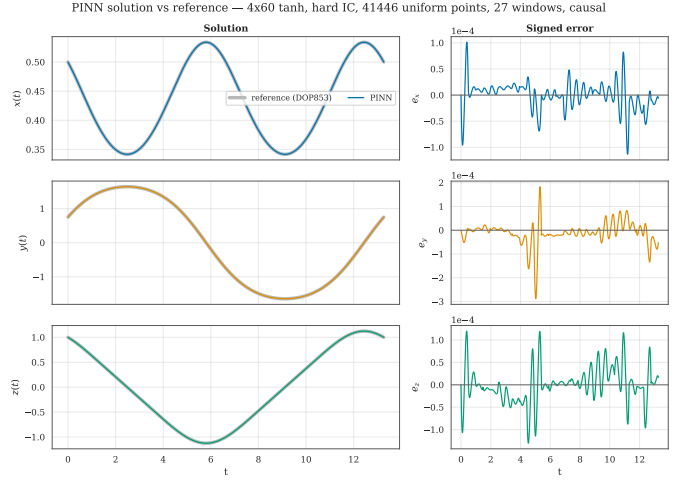


Fig. 3. PINN solution vs reference with signed errors.

TABLE IV
SAVED FULL-INTERVAL AND WINDOWED RUNS ON THE SAME IVP

Measure	One network	27 windows
Trainable parameters	11,283	304,641
Training points	39,793 LHS	41,446 uniform
Combined state RMSE	2.237	6.97×10^{-5}
Maximum state-error norm	3.346	3.09×10^{-4}
Collocation residual MSE	2.80×10^{-4}	1.87×10^{-7}
Recorded Adam steps	40,000	34,765

Lastly, the hard trial solution ensures that the values of the state are exactly continuous at the inner 26 interfaces connecting the windows.

B. Run Comparison

The differences with respect to the two saved configurations are shown in Table IV. The RMSE value of the entire interval network is 2.237, which is much larger than the windowed run network’s RMSE value of 6.97×10^{-5} . It is a notable result for the recorded settings of this initial value problem. This is a comparison of two saved configurations, which differ in causal weighting [3], windowing [6] and warm starts as well as precision and even L-BFGS. In particular the model with the window is much more complicated with 27 times as many parameters and a different collocation grid.

The 27 run windows had 34,765 Adam updates and 33,143 L-BFGS closure evaluations in total. Closure evaluations are function calls made by L-BFGS, not iterations. Only two of the 108 causal stages ended at the 4,000-step cap without meeting the 0.99 weight threshold.

C. Ablation and Failure Results

Table I shows the ablation results. The median RMSE of the candidate is 8.13×10^{-5} , and three of the five seeds are below 10^{-4} . Without causal weights the median is about 2.3 times larger, but for seed 2 the no-causal run has lower RMSE than the candidate. Without warm start, all three seeds have larger RMSE than the candidate with the same seed. The mean

Adam steps also rise to 102,989, and 17–19 of the 108 stages reach the cap, while the candidate has only 1–2. When one network is reused for all windows, the RMSE is between 0.350 and 0.648. With separate networks and the same no-causal schedule it is between 1.75×10^{-4} and 5.16×10^{-4} . This matches what we noticed in Section III, that the previous learning is lost or degraded when one network moves to a new window. The causal single network gives 0.275 in its one run. This is lower than the full-interval run but still far from the windowed runs. These runs support the combination of windows, warm start and causal weights that we tested for this orbit. But with three to five seeds, they do not show that causal weighting always improves the accuracy.

VI. CONCLUSION

Each of our 27 window PINN passes each of the tests on the nearly closed Lorenz-system interval $[0, 13.26446]$. Each window starts from the predicted endpoint of the previous window. The seed-0 run had a combined state RMSE of 6.97×10^{-5} in comparison to DOP853, and the median over five seeds is 8.13×10^{-5} . A network designed separately to full intervals recorded an RMSE of 2.237. This research applies to only one initial state and one coefficient set. Other initial states and other coefficient sets may result in different behavior. Future research will focus on matched configurations and initial states and generalise to problems with specified boundary conditions at each end.

REFERENCES

- [1] E. N. Lorenz, “Maximum simplification of the dynamic equations,” *Tellus*, vol. 12, no. 3, pp. 243–254, 1960.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [3] S. Wang, S. Sankaran, and P. Perdikaris, “Respecting causality for training physics-informed neural networks,” *Computer Methods in Applied Mechanics and Engineering*, vol. 421, p. 116813, 2024.
- [4] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [5] J. Matthews and A. Bihlo, “Pinnde: physics-informed neural networks for solving differential equations,” *arXiv preprint arXiv:2408.10011*, 2024.
- [6] M. Penwarden, A. D. Jagtap, S. Zhe, G. E. Karniadakis, and R. M. Kirby, “A unified scalable framework for causal sweeping strategies for physics-informed neural networks (PINNs) and their temporal decompositions,” *Journal of Computational Physics*, vol. 493, p. 112464, 2023.
- [7] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.